

CAREERBRIDGE – DAY 5 EXERCISE

Topic: Git, GitHub and Week 1 Project Review

Project: CareerBridge – Job & Internship Portal

Week: 1

Day: 5

Student Name: Larbi Peniels Okyerewa

Date: 09/13/2026

1. OBJECTIVES

The purpose of today's exercise is to introduce professional source control using Git and GitHub and to review everything completed during Week 1.

By the end of today, you should be able to:

- Explain what Git and GitHub are.
 - Create a Git repository.
 - Connect a local repository to GitHub.
 - Use basic Git commands.
 - Create meaningful commits.
 - Push changes to GitHub.
 - Understand the purpose of .gitignore.
 - Organize CareerBridge project files.
 - Present and explain the work completed during Week 1.
-

PART A — GIT FUNDAMENTALS

Exercise 1 — Explain the following terms

Write a short explanation in your own words.

Git

It is a version control system which tracks changes in your project.

GitHub

It is an online platform that allows you to repositories and also share them.

Repository

It is your project with your git history.

Commit

Records the changes you make in your project.

Branch

It is a separate version of your project.

Push

Sends changes you make on your computer to GitHub.

Pull

Sends changes you make in your GitHub to your computer.

Clone

Making a copy of your repository from GitHub into your computer.

PART B — CREATE THE CAREERBRIDGE REPOSITORY

Create a new GitHub repository called:

careerbridge

The repository should be used for the CareerBridge project throughout the attachment.

Repository Checklist

- ☐ Repository created
 - ☐ Repository name is careerbridge
 - ☐ Repository has a meaningful description
 - ☐ README file created
 - ☐ .gitignore created
 - ☐ Local project connected to GitHub
-

PART C — ORGANIZE THE PROJECT

Create a clear project structure.

A possible starting structure is:

careerbridge/

|

|— database/

| └─ CareerBridge_Database.sql

|

|— docs/

| └─ requirements/

| └─ wireframes/

| └─ erd/

|

|— backend/

|

|— frontend/

```
|
|— .gitignore
|
|— README.md
```

The backend and frontend folders may remain empty for now.

They will be used in Weeks 2 and 3.

You may improve the structure if you have a good reason.

PART D — INITIALIZE GIT

Open a terminal inside the CareerBridge project directory.

Initialize Git.

```
git init
```

Check the current repository status.

```
git status
```

Write below what git status tells you.

PART E — ADD FILES TO GIT

Add your project files to Git.

```
git add .
```

Run:

```
git status
```

Question

What changed after running git add .?

After running `git add .`, the files that were previously untracked or modified were moved to the staging area. Git status showed that the changes were ready to be included in the *commit*.

What does the term **staging area** mean?

The staging area is the place where Git keeps the changes you have selected to be included in the next commit.

PART F — CREATE YOUR FIRST COMMIT

Create a commit.

Do not use messages such as:

update

changes

stuff

work

test

Commit messages should describe what changed.

For example:

```
git commit -m "Add initial CareerBridge project structure"
```

Write your first commit message:

```
git commit -m "Add initial CareerBridge project structure"
```

PART G — CONNECT TO GITHUB

Connect your local repository to the GitHub repository you created.

Follow the GitHub instructions for connecting an existing local repository.

After connecting it, push your work.

Your final command may look similar to:

```
git push -u origin main
```

Open the GitHub repository in your browser and confirm that the files are visible.

Verification

- ☐ Files appear on GitHub
 - ☐ README is visible
 - ☐ Database script is visible
 - ☐ Documentation folders are visible
 - ☐ Commit history is visible
-

PART H — PRACTISE THE GIT WORKFLOW

Create a small change in your README.md.

For example, add a section describing the technologies planned for CareerBridge.

Then perform the complete workflow:

Modify



git status



git add



git commit



git push

Do not simply copy the commands.

Observe what happens at each stage.

PART I — README

Create a useful README.md for CareerBridge.

It should contain at least:

CareerBridge

Project Description

Project Objectives

User Types

Main Features

Technology Stack

Project Structure

Current Development Status

Under **Current Development Status**, indicate that Week 1 focused on:

- Requirements analysis
- User stories
- Navigation
- Wireframes
- ERD
- Database design
- SQL practice

Do not copy large sections of the requirements document into the README.

Keep it concise.

PART J — .GITIGNORE

Create a .gitignore file.

Research what .gitignore is used for.

Answer:

What is the purpose of .gitignore?

It is used to tell the files that should not be tracked or uploaded to GitHub

Why should .env normally not be committed to GitHub?

.env files should not be committed because they may contain sensitive information.

Why should node_modules normally not be committed?

It contains the dependencies used by the Node.js project. It can be very large so uploading it would increase the size of the repository

We will use these concepts when backend development begins.

PART K — GIT COMMAND CHALLENGE

Without looking at the examples above, write the Git command you would use for each task.

1. Check the current repository status

`git status`

2. Add all changed files

`git add .`

3. Commit your changes

`git commit -m "Update CareerBridge project"`

4. Display previous commits

`git log`

5. Push committed changes to GitHub

`git push`

6. Download an existing GitHub repository to your computer

`git clone <repository-link>`

7. Download recent changes from the remote repository

`git pull`

PART L — UNDERSTANDING SOURCE CONTROL

Answer the following questions in your own words.

1. Why is Git useful when developing software?

It allows you to track changes made in your project.

2. What is the difference between Git and GitHub?

Git runs on a computer and track changes to a project whiles GitHub is an online platform that allows you to store git repositories and share it.

3. What is the difference between `git add` and `git commit`?

`git add` is used to add changes made in a project whiles `git commit` is used to save those changes.

4. What is the difference between `git commit` and `git push`?

`Git commit` is a saves snapshot of changes in a repository whiles `git push` allows you to send those changes to your GitHub.

5. Why are meaningful commit messages important?

They explain what was changed in each commit.

6. What could happen if passwords or database credentials are committed to a public GitHub repository?

They could be exposed to unauthorized people.

7. Why is Git useful when several developers are working on the same project?

The various part that each developer is working on can be added together.